VEON Best-Practice Örnek Uygulaması — Yönetici Özeti

Nov 12, 2026 · @cihan demir

Özet

Mevcut agentic-core kodunda kullanıcı gecikmesini, veri tutarlılığını ve kesinti süresini doğrudan etkileyen 6 yüksek öncelikli risk tespit edildi. Bu risklerin çözümünü gösteren, çalışan ve ölçülmüş bir referans uygulama hazırlandı; geliştiriciler aynı desenleri core'a adım adım taşıyabilir.

- **Redis kesintisinde servis ayakta kalıyor:** ana sunucu çöktüğünde ~2 sn kesinti ve otomatik geçiş; kontrollü geçişte 0 olay kaybı.
- **Eşzamanlı yükte veri tutarlı:** 10 stoklu kampanyada 100 eşzamanlı alıcıdan tam 10 satış (mevcut desenle 100 satış).
- **Log ve denetim kaydı kullanıcı isteğinden ayrıldı:** arka plan işçileri dururken bile API gecikmesi değişmedi; biriken 10.000 olay 1,6 sn'de yazıldı.
- **Bir isteğin izi uçtan uca tek ekranda:** API, Redis, kuyruk, işçi ve MongoDB adımları tek kayıtta (Grafana Tempo ve Jaeger).

Uygulama Docker ve Helm ile paketli ve mevcut VEON izleme altyapısıyla (Grafana, Prometheus, Loki, Tempo) uyumludur.

Dikkat çekilen noktalar

En kritik bulgu, olayların kullanıcı isteğinin içinde MongoDB'ye yazılması: veritabanındaki her yavaşlama doğrudan kullanıcıya yansıyor. 09.11.2026 tarihli kaynak kodu incelendi; canlı ortamda doğrulama yapılmadı.

#	Bulgu	İş etkisi	Öncelik
1	Olaylar kullanıcı isteği içinde MongoDB'ye yazılıyor	MongoDB yavaşlarsa kullanıcı da yavaşlar; kesintide olaylar kaybolur	Yüksek
2	Aynı olay iki kez kaydedilebiliyor (benzersizlik kontrolü yok)	Raporlarda çift kayıt; denetim verisi güvenilirliği düşer	Yüksek
3	Yetki önbellek anahtarı her sunucuda farklı üretiliyor	Önbellek sunucular arasında paylaşılmıyor; yetki servisine gereksiz yük	Yüksek

#	Bulgu	İş etkisi	Öncelik
4	Oturum (session) güncellemesi atomik değil	Eşzamanlı işlemlerde konuşma durumu sessizce kaybolabilir	Yüksek
5	Oturum kayıtlarının Redis'te süresi yok	Okunmayan oturumlar birikir; bellek dolunca yazmalar reddedilir	Yüksek
6	Redis bağlantılarında zaman aşımı ve yeniden deneme yok	Ağ sorununda istekler asılı kalır; Redis geçişinde hata alınır	Yüksek
7	Kapanışta hata (senkron metoda await)	Deploy ve yeniden başlatmada temiz kapanmama	Orta
8	Metriklerde ham URL kullanımı	İzleme maliyeti artar; Prometheus kararlılığı riske girer	Orta
9	İstek başına iki log satırı; sorgu parametresi ve IP loglanıyor	Log maliyeti iki katı; kişisel veri (KVKK) riski	Orta
10	Tek sağlık kontrolü her zaman "ok" dönüyor	Bozulmuş sunucu trafik almaya devam eder	Orta
11	MongoDB sürücüsü (Motor) Mayıs 2026'da destek dışı kaldı	Güvenlik yaması alınamaz	Orta

Korunması gereken güçlü yanlar: hassas veri maskeleme, Redis Sentinel desteği ve RabbitMQ mesajlarındaki tekrar koruması.

Ne yapıldı

Beş teknik konu, tek bir örnek uygulamada (ürün kataloğu ve kampanya senaryosu) birlikte gösterildi; ayar isimleri mevcut core ile aynı tutuldu.

Alan	Ne sağlar
İzleme (tracing)	Bir isteğin tüm adımları tek kayıtta; Grafana Tempo ve Jaeger arayüzlerinden görülür
İzleme (monitoring)	Hata oranı, gecikme ve kuyruk birikmesi metrikleri; 12 alarm kuralı; hazır dashboard
Loglama	Tek satır yapılandırılmış loglar; Loki ve Elasticsearch/Kibana (EFK); kişisel veri maskeleme

Alan	Ne sağlar
Redis yüksek erişilebilirlik	1 ana + 2 yedek sunucu, 3 gözcü (Sentinel); otomatik geçiş; önbellek, kilit, hız sınırı ve tekrar koruması desenleri
Asenkron log ve olay yazımı	Kuyruk + arka plan işçileri; kayıp ve çift kayıt koruması
Paketleme	Docker imajları (yetkisiz kullanıcı, salt okunur); Helm chart (otomatik ölçekleme, ağ politikası, izleme entegrasyonu)
Lab arayüzü	Tüm senaryolar tek ekranda, "doğru" ve "yanlış" desen karşılaştırmalı

Her konu için geliştiricilere yönelik ayrıntılı rehber (docs/01–07) ve otomatik testler (11 test + uçtan uca smoke testi) mevcut.

Ölçülen sonuçlar

Doğru desenler, aynı yük altında veritabanı yükünü 50 kat azalttı ve veri tutarsızlığını sıfırladı. Ölçümler yerel Docker ortamında, 2 API sunucusu ve 2 arka plan işçisiyle yapıldı; üretim değerleri ağ ve donanıma göre değişir.

Senaryo	Sonuç	İş açısından anlamı
Boş önbelleğe 50 eşzamanlı istek	Veritabanına 1 istek (korumasız: 50)	Trafik patlamasında veritabanı çökmez
10 stoklu kampanya, 100 eşzamanlı alıcı	Tam 10 satış (mevcut desen: 100)	Fazla satış ve tutarsız bakiye olmaz
Aynı isteğin 11 kez tekrarı	İşlem yalnız 1 kez yapıldı	Çift tahsilat ve çift kayıt olmaz
30 istek, limit 10	Tam 10 izin, 2 sunucu genelinde	Kötüye kullanıma karşı tutarlı koruma
Redis ana sunucusu çöktü	~2 sn kesinti, otomatik geçiş	Elle müdahale gerekmez
Redis ana sunucusu dondu (ağ kopması)	~2–3 sn kesinti; istekler asılı kalmadı	Kullanıcı uzun bekleme yaşamaz
Kontrollü Redis geçişi (bakım)	~9 sn yazma reddi, 0 olay kaybı	Bakım sırasında veri kaybolmaz

Senaryo	Sonuç	İş açısından anlamı
İşçiler durdu, 10.000 olay üretildi	API gecikmesi değişmedi; işçiler dönünce 1,6 sn'de yazıldı	Arka uç arızası kullanıcıya yansımaz
Tek rezervasyon isteği	6 adım, 2 servis tek iz kaydında	Arızanın kaynağı tek ekrandan bulunur

Önerilen değişiklikler ve etkileri

İlk üç değişiklik düşük eforla en yüksek riski kapatır ve mevcut davranışı değiştirmez; bu yüzden Faz 1'de önerilir. Her değişikliğin çalışan karşılığı örnek uygulamada mevcut.

#	Değişiklik	Etkisi	Efor	Geçiş riski	Faz
1	Yetki önbellek anahtarını sabit bir özetle (HMAC) üret	Önbellek tüm sunucularda paylaşılır; yetki servisinin yükü düşer	Düşük	Düşük: geçişte önbellek bir kez boşalır	Faz 1
2	Redis bağlantılarına zaman aşımı, yeniden deneme ve havuz sınırı	Asılı kalan istek olmaz; Redis geçişi otomatik atlatılır	Düşük	Düşük	Faz 1
3	Kapanış hatasını düzelt; sağlık kontrolünü ikiye ayır	Temiz deploy; bozulan sunucu trafikten otomatik çıkar	Düşük	Düşük	Faz 1
4	Metrik ve log düzeltmeleri (şablon URL, tek satır log, iz kimliği)	İzleme maliyeti düşer; KVKK riski azalır; logdan ize tek tıkla geçiş	Düşük	Orta: dashboard ve alarm sorguları güncellenmeli	Faz 2
5	Oturumlara Redis süresi ve atomik güncelleme	Bellek sızıntısı ve kayıp güncelleme sona erer	Orta	Orta: eşzamanlılık testleri gerekir	Faz 2
6	Olay yazımını asenkron hatta taşı; benzersizlik indeksi ekle	MongoDB yavaşlığı kullanıcıya yansımaz; çift kayıt biter	Orta	Orta: gölge yazımla karşılaştırmalı geçiş	Faz 3
7	Motor yerine PyMongo Async; Redis veritabanı numarası yerine anahtar öneki	Desteklenen sürücü; Redis Cluster'a geçiş yolu açılır	Yüksek	Yüksek: anahtar taşıma planı gerekir	Faz 3

Efor değerleri göreli tahmindir; kesin süreler takım planlamasında netleşir.

Yol haritası

Geçiş, riski en düşükten yükseğe üç fazda yapılır; bir faz, çıkış kriteri stage ortamında karşılanmadan kapanmaz.

1. **Faz 1 — Hızlı kazanımlar** (değişiklik 1–3): mevcut davranışı değiştirmeyen düzeltmeler.
    - Çıkış kriteri: stage'de Redis geçiş tatbikatı yapıldı, asılı kalan istek yok, yetki önbelleği isabet oranı sunucu sayısından bağımsız.
2. **Faz 2 — Gözlemlenebilirlik ve oturumlar** (değişiklik 4–5).
    - Çıkış kriteri: dashboard ve alarmlar yeni metriklerle çalışıyor; eşzamanlılık testleri yeşil; Redis bellek kullanımı sabit.
3. **Faz 3 — Mimari geçişler** (değişiklik 6–7).
    - Çıkış kriteri: gölge yazımda eski ve yeni hat kayıt sayıları eşleşiyor; kayıp ve çift kayıt sıfır; anahtar taşıma geri alınabilir planla tamamlandı.

Riskler, takaslar ve sınırlar

En önemli takas, Redis geçişinde veri güvenliği için kabul edilen ~9 saniyelik yazma kesintisidir.

- **Dayanıklılık ve erişilebilirlik takası:** kontrollü geçişteki ~9 sn yazma reddi, ağ bölünmesinde eski sunucuya yazılıp veri kaybedilmesini önlemek için bilinçli bir ayardır. Kapatılırsa kesinti kısalır, veri kaybı riski doğar.
- **Redis kalıcı kaynak değildir:** ana sunucu çökerken son milisaniyelerdeki yazmalar kaybolabilir. Kaybı kabul edilemeyen veri MongoDB veya RabbitMQ'da tutulmalıdır.
- **Bellek kuyruğu sınırı:** sunucu ani çökerse kuyrukta bekleyen (1 sn'den kısa) olaylar kaybolabilir. Kayıp hiç kabul edilemeyen olaylar için "outbox" deseni önerilir.
- **Doğrulama kapsamı:** Helm chart gerçek bir Kubernetes kümesine kurulmadı; yalnız şablon ve şema doğrulamasından (66/66 kaynak) geçti. KEDA tetikleyicisi ve Bitnami Redis örneği çalışırken denenmedi.
- **İmaj kaynağı:** Bitnami'nin ücretsiz imaj kataloğu 2025'te değişti; Redis için kurum içi onaylı imaj kaynağı belirlenmelidir.
- **Altyapı maliyeti:** Jaeger ve Elasticsearch kalıcı kullanımda ayrı depolama ve bellek ister; yerel ortamda Elasticsearch tek başına ~1 GB RAM kullandı.
- **Yerel ortam:** MongoDB kimlik doğrulamasız ve Grafana anonim erişimli çalışır; bu ayarlar yalnız demo içindir.

Karar gerektiren konular

Faz 1 hemen başlayabilir; aşağıdaki kararlar Faz 2 ve 3'ün kapsamını belirler.

- [ ] **İz arayüzü standardı:** yalnız Tempo (VEON standardı) mı, Tempo ile birlikte Jaeger mi?
- [ ] **Log hedefi:** yalnız Loki mi, uzun saklama ve analitik için ek olarak Elasticsearch mi?
- [ ] **Redis ayrımı:** önbellek ile kuyruk/kilit gibi kalıcı veriler için ayrı Redis kurulumları açılsın mı?
- [ ] **Geçiş davranışı:** Redis geçişinde veri güvenliği öncelikli ayar (~9 sn yazma kesintisi) mi, daha kısa kesinti mi?
- [ ] **Otomatik ölçekleme:** arka plan işçilerini kuyruk birikmesine göre ölçeklemek için KEDA kurulsun mu?
- [ ] **Sahiplik ve takvim:** her faz için sorumlu ekip ve hedef tarih.

Ek: proje ve dokümanlar

Proje klasörü `veon-best-practice-sample`; giriş noktası README.md. Ekip erişimi için kurum içi Git deposuna alınması önerilir.

Doküman	İçerik
README.md	Mimari, hızlı başlangıç, ölçülen sonuçlar, üretime geçiş kontrol listesi
docs/01-tracing.md	İz kaydı (OpenTelemetry), Tempo ve Jaeger
docs/02-monitoring.md	Metrikler, alarmlar, dashboard
docs/03-logging.md	Yapılandırılmış loglama, Loki ve EFK karşılaştırması
docs/04-redis-ha-concurrency.md	Redis yüksek erişilebilirlik ve eşzamanlılık desenleri, geçiş ölçümleri
docs/05-async-log-pipeline.md	Asenkron olay hattı, garantiler ve arıza senaryoları
docs/06-kubernetes-helm.md	Docker imajları ve Helm chart
docs/07-agentic-core-uyum-notlari.md	Mevcut koddaki bulgular, satır referansları ve önerilen geçiş sırası

Yerelde çalıştırma: proje klasöründe `make up`, ardından Lab arayüzü http://localhost:8080.